DEPLOYADA

RELEASE 2 · SPRINT 02

Kubernetes

na prática: a ordem por escrito ♥

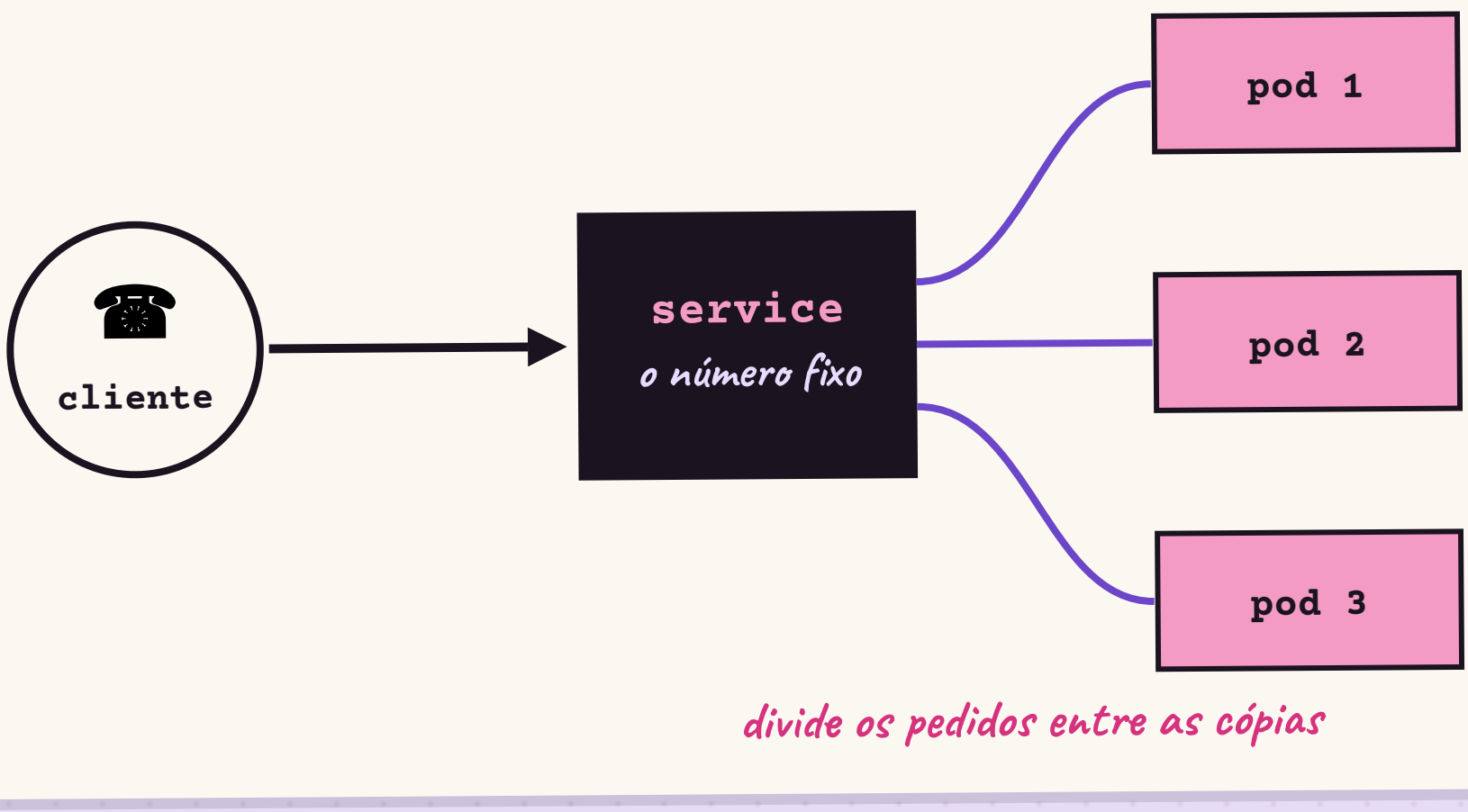


material de apoio da sprint

@deployada

O telefone fixo da rede

Os pods vêm e vão. O **service** é o número que nunca muda.



1. O cliente liga sempre pro **mesmo endereço**.
2. O service repassa pra uma sacola disponível e **divide a carga**.

A ordem por escrito

Salve como **deployment.yaml**. É o manifesto da missão da sprint.

```yaml
apiVersion: apps/v1
kind: Deployment          # a ordem de serviço
metadata:
  name: meu-site
spec:
  replicas: 3             # quero sempre 3 sacolas
  selector:
    matchLabels:
      app: meu-site
  template:               # como é cada sacola
    metadata:
      labels:
        app: meu-site
    spec:
      containers:
      - name: web
        image: nginx:1.27     # a marmita
        ports:
        - containerPort: 80
```

1. Entregue pro gerente: **kubectl apply -f deployment.yaml**
2. Os espaços importam no YAML: copie igualzinho.

O dia a dia da ordem

Com a ordem no arquivo, **mudar é editar e aplicar de novo**.

↑

Mais movimento

Troque replicas: 3 por 5 e aplique de novo. Ou deixe automático, pelo uso de CPU.

⇄

Receita nova

Mude a versão da imagem e aplique. O gerente troca as sacolas aos poucos, sem fechar a loja.

↶

Deu errado?

kubectl rollout undo volta pra versão anterior.

✦

O manifesto vai pro Git

Junto com o código. Guarda essa ideia pra Sprint 03.

E QUANDO NÃO USAR?

Projeto pessoal ou sistema pequeno? Docker Compose resolve. Kubernetes num site de três páginas é contratar gerente de rede pra uma barraquinha de pastel.

Colinha da prática

~/rede-de-restaurantes — bash

`kubectl apply -f arquivo.yaml`	Entrega (ou atualiza) a ordem por escrito.
`kubectl get pods`	Lista as sacolas e mostra se estão rodando.
`kubectl get services`	Lista os telefones fixos.
`kubectl expose deployment nome`	Coloca um service na frente. `--type=LoadBalancer --port=8080 --target-port=80`
`kubectl scale deployment nome`	Muda quantas cópias, sem editar o arquivo. `--replicas=5`
`kubectl rollout status deployment nome`	Acompanha a troca de versão.
`kubectl rollout undo deployment nome`	Volta pra versão anterior.
`kubectl describe pod nome`	Conta por que um pod caiu.
`kubectl delete -f arquivo.yaml`	Apaga tudo que o arquivo criou.

Deu ruim em produção

🔥 O CASO

Black Friday. O acesso explode, os pods passam do limite de memória e o Kubernetes derruba cada um (`OOMKilled`). Sobe outro, que cai de novo: `CrashLoopBackOff`. Site fora do ar.

o gerente cumpriu a ordem. a ordem é que estava errada!

1 **Descubra o porquê**
kubectl describe pod e kubectl logs contam por que o pod morreu.

2 **Limite medido, não chutado**
Defina memória e CPU a partir do uso real da aplicação.

3 **Teste com carga antes**
Simule o pico antes da data grande, não durante.

```
# no container do deployment
resources:
  requests:
    memory: "256Mi"  # o que ele precisa
  limits:
    memory: "512Mi"  # o máximo da sacola
```

Missão da sprint

Entregue a sua **primeira ordem por escrito**, com o Kubernetes do Docker Desktop ligado.

1 Salve o manifesto da página 3 como `deployment.yaml`.

2 Rode `kubectl apply -f deployment.yaml` e confira com `kubectl get pods`.

3 No arquivo, troque `replicas: 3` por `5`, salve e rode o apply de novo.

4 Coloque o telefone fixo:

```
kubectl expose deployment meu-site
--type=LoadBalancer --port=8080 --target-port=80
```

5 Abra `localhost:8080`: o nginx aparece!

Pra limpar: `kubectl delete -f deployment.yaml` e `kubectl delete service meu-site`

kubectl

até o próximo deploy!

NA SPRINT 03

CI/CD e GitOps, a esteira da cozinha

Ouça no Spotify, no Deezer ou direto pelo site.
Tudo no link da bio do **@deployada**

Release 2